Rockfish has Ranked #496 in top500.org's June 2024 Top 500 List (https://www.arch.jhu.edu/top500-june2024/)

Submit a Help Ticket (https://www.arch.jhu.edu/help-ticket/)

Search...

JOHNS HOPKINS UNIVERSITY | Advanced Research Computing at Hopkins

(https://www.arch.jhu.edu/)

Contact(https://www.arch.jhu.edu/help-ticket/)
ARCH Systems Documentation(https://docs.arch.jhu.edu/en/latest/)
Cluster Status(https://status.arch.jhu.edu/status/arch)

How to create a Slurm script

Slurm scripts are used to submit and manage jobs in a high-performance computing (HPC) environment that uses the Slurm workload manager. Slurm is a popular open-source resource management and job scheduling application used on many HPC clusters and supercomputers.

A basic example of a Slurm script

```
#!/bin/bash
#SBATCH --job-name=my_job_name        # Job name
#SBATCH --output=output.txt           # Standard output file
#SBATCH --error=error.txt             # Standard error file
#SBATCH --partition=partition_name    # Partition or queue name
#SBATCH --nodes=1                     # Number of nodes
#SBATCH --ntasks-per-node=1           # Number of tasks per node
#SBATCH --cpus-per-task=1             # Number of CPU cores per task
#SBATCH --time=1:00:00                # Maximum runtime (D-HH:MM:SS)
#SBATCH --mail-type=END               # Send email at job completion
#SBATCH --mail-user=your@email.com    # Email address for notifications

#Load necessary modules (if needed)
#module load module_name

#Your job commands go here
#For example:
#python my_script.py

#Optionally, you can include cleanup commands here (e.g., after the job finish
#For example:
#rm some_temp_file.txt
```

Here's an explanation of the key Slurm directives in the script:

- **#SBATCH** These lines are comments in a Slurm script and specify various options for the job.
- **–job-name** A name for your job.
- **–output** and **–error:** The paths to the standard output and error log files.
- **–partition:** The name of the Slurm partition or queue where the job should run.
- **–nodes:** The number of nodes needed for the job.
- **–ntasks-per-node:** The number of tasks per node or processes to run.
- **–cpus-per-task:** The number of CPU cores allocated to each task.
- **–time:** The maximum runtime for the job.

- **–mail-type** and **–mail-user:** Email notification settings.

> Note
>
> Please avoid to use `--ntasks` on Rockfish also, no need to set `--mem` It will automatically set to the number of cores, 4GB per core.

After the `#SBATCH` directives, you can load any necessary modules or execute your job's commands. In the example, it's assumed that you will run a Python script named `my_script.py`. You can replace this with your specific job commands. To submit a Slurm job, you can save the script to a file (e.g., `my_job.slurm`) and then use the `sbatch` command to submit the job:

```
[userid@local ~]$ sbatch my_job.slurm
```

> Note
>
> The provided script is a Slurm job script written in Bash for submitting a job array to a Slurm cluster. Here's a breakdown of the script:

How to run a matlab job array

This script is designed to run a job array, where a job is executed 20 times with different values of `$SLURM_ARRAY_TASK_ID`, which ranges from 1 to 20.

```
#!/bin/bash -l
#SBATCH --job-name=job-array2        # Job name
#SBATCH --time=1:1:0                 # Maximum runtime (D-HH:MM:SS)
#SBATCH --array=1-20                 # Defines a job array from task ID 1 to 2
#SBATCH --ntasks=1                   # Number of tasks (in this case, one task
#SBATCH -p shared                      # Partition or queue name
#SBATCH --reservation=Training       # Reservation name
#SBATCH                              # This is an empty line to separate Slurm

#run your job

echo "Start Job $SLURM_ARRAY_TASK_ID on $HOSTNAME"  # Display job start inform

sleep 10  # Sleep for 10 seconds

export alpha=1  # Set an environment variable alpha to 1
export beta=2   # Set an environment variable beta to 2

module load matlab  # Load the Matlab module

matlab -nodisplay -singleCompThread -r "myRand($SLURM_ARRAY_TASK_ID, $alpha, $
#Run a Matlab script with parameters: $SLURM_ARRAY_TASK_ID, $alpha, and $beta,
```

Here’s what the script does:

- The script specifies Slurm directives at the beginning of the file. These directives provide instructions to the Slurm scheduler for managing the job array, such as the job name, maximum runtime, array definition, number of tasks, partition, and reservation.
- After the Slurm directives, the script contains actual job commands. It starts by echoing a message indicating the start of the job with the current task ID and the hostname where the job is running.
- It then `sleeps` for 10 seconds using the sleep command.
- Two environment variables, `alpha` and `beta`, are exported with values 1 and 2, respectively.
- The Matlab module is loaded with the `module load` command.
- Finally, Matlab is invoked with the specified parameters using the `mpirun` flag. The `MyRand` Matlab function is called with the current `$SLURM_ARRAY_TASK_ID`, `$alpha`, and `$beta`. It also includes a pause(20) to pause execution for 20 seconds and then exits.

To submit this job array script to the Slurm scheduler, save it to a file (e.g., `job_array_script.sh`) and then submit it using the `sbatch` command:

```
[userid@local ~]$ sbatch job_array_script.sh
```

Note

The scheduler will take care of running the job array with the specified parameters.

How to run job array task with a step size☒

When using **#SBATCH –array=1-100%10**, it defines a job array where the task IDs range from 1 to 100, and each job array element runs every 10 task IDs. This means that you will have a total of 10 job instances, each running a subset of the task IDs from 1 to 100. Here's an example script using this array configuration:

```
#!/bin/bash -l
#SBATCH --job-name=job-array-example
#SBATCH --time=1:0:0
#SBATCH --array=1-100%10  # Job array from task ID 1 to 100, with a step size
#SBATCH --ntasks-per-node=1
#SBATCH --partition=shared
#SBATCH --mail-type=end
#SBATCH --mail-user=userid@jhu.edu
#SBATCH --reservation=Training

ml intel/2022.2

#Your executable or script goes here
#Example: Running a Python script
#python my_script.py $SLURM_ARRAY_TASK_ID

#In this example, each job instance will execute the script with a different S
```

In this script:

- `-#SBATCH --array=1-100%10` defines a job array with task IDs ranging from 1 to 100, where each job instance will run a subset of 10 consecutive task IDs. So, you'll have 10 job instances with `SLURM_ARRAY_TASK_ID` values like 1, 11, 21, ..., 91.
- The `ml intel/2022.2` line loads the Intel compiler module, which can be used for compilation if your job requires it.
- The actual job commands, such as running an executable or script, should be placed below the comments. In this example, I've left a placeholder comment indicating how you might run a Python script with the `SLURM_ARRAY_TASK_ID`. You should replace it with the actual commands or scripts you want to execute for your job.

To submit this job array to the Slurm scheduler, save it to a file (e.g., `job_array_example.sh`) and then submit it using the sbatch command:

```
[userid@local ~]$ sbatch job_array_example.sh
```

Note

The scheduler will create 10 job instances, each running a subset of task IDs according to the specified array configuration.

How to run an MPI (Message Passing Interface) program

To perform a Slurm job script for running an MPI (Message Passing Interface) program on a high-performance computing (HPC) Rockfish Cluster.

Here's a breakdown of the script:

```
#!/bin/bash -l
#SBATCH --job-name=mpi-job          # Job name
#SBATCH --time=1:0:0                # Maximum runtime (1 hour)
#SBATCH --nodes=1                   # Number of nodes requested
#SBATCH --ntasks-per-node=4         # Number of MPI tasks per node
#SBATCH --partition=shared            # Partition or queue name
#SBACTH --mail-type=end             # Email notification type (end of job)
#SBATCH --mail-user=userid@jhu.edu  # Email address for notifications
#SBATCH --reservation=Training      # Reservation name

ml intel/2022.2  # Load the Intel compiler module with version 2022.2

# compile
mpiicc -o hello-mpi.x hello-mpi.c  # Compile the MPI program from source code

mpirun -np 4 ./hello-mpi.x > my-mpi.log  # Run the MPI program with 4 MPI proc
```

Here's what the script does:

1. It specifies various Slurm directives at the beginning of the script. These directives provide instructions to the Slurm scheduler for managing the MPI job:

- **–job-name** Specifies a name for the job.
- **–time** Sets the maximum runtime for the job to 1 hour.
- **–nodes** Requests 1 compute node for the job.
- **–ntasks-per-node** Specifies that there will be 4 MPI tasks per node.
- **–partition** Specifies the Slurm partition or queue where the job should run (in this case, `shared`).

- **–mail-type** Requests email notifications at the end of the job.
- **–mail-user** Specifies the email address where notifications will be sent.
- **–reservation** Associates the job with a reservation named “Training.”

2. The script loads the Intel compiler module with version 2022.2 using the `ml` command. This is done to ensure that the correct compiler environment is set up for compilation.
3. It compiles the MPI program named `hello-mpi.c` using the `mpiicc` compiler and generates an executable named “hello-mpi.x.”
4. Finally, it runs the MPI program using the mpirun command with 4 MPI processes. The standard output of the program is redirected to a log file named `my-mpi.log`.

To submit this MPI job to the Slurm scheduler, save it to a file (e.g., `mpi_job_script.sh`) and then submit it using the sbatch command:

```
[userid@local ~]$ sbatch mpi_job_script.sh
```

Note

The scheduler will allocate resources and run the MPI program with the specified parameters.

How to run a mixed MPI / OpenMP program

To submit a Slurm job script for running a mixed MPI/OpenMP program on a high-performance computing (HPC) cluster. This script combines both message-passing parallelism (MPI) and shared-memory parallelism (OpenMP). Here’s a breakdown of the script:

```
#!/bin/bash -l
#BATCH --job-name=omp-job          # Job name
#SBATCH --time=1:0:0                # Maximum runtime (1 hour)
#SBATCH --nodes=2                   # Number of nodes requested
#SBATCH --ntasks-per-node=1         # Number of MPI tasks per node
#SBATCH --cpus-per-task=4           # Number of CPU cores per task
#SBATCH --partition=shared          # Partition or queue name
#SBACTH --mail-type=end             # Email notification type (end of job)
#SBATCH --mail-user=$USER@jhu.edu   # Email address for notifications (using t
#SBATCH --reservation=Training      # Reservation name

ml intel/2022.2  # Load the Intel compiler module with version 2022.2

#Compile the code using Intel and mix MPI/OpenMP
echo "mpiicc -qopenmp -o hello-mix.x hello-world-mix.c"

#How to compile
#mpiicc -qopenmp -o hello-mix.x hello-world-mix.c

#Run the code
mpirun -np 2 ./hello-mix.x  # Run the mixed MPI/OpenMP program with 2 MPI proc
```

Here's what the script does:

1. The script specifies various Slurm directives at the beginning of the script. These directives provide instructions to the Slurm scheduler for managing the mixed MPI/OpenMP job:

- **–job-name** Specifies a name for the job.
- **–time** Sets the maximum runtime for the job to 1 hour.
- **–nodes** Requests 2 compute nodes for the job.
- **–ntasks-per-node** Specifies that there will be 1 MPI task per node.
- **–cpus-per-task** Specifies that each MPI task will use 4 CPU cores.
- **–partition** Specifies the Slurm partition or queue where the job should run (in this case, "shared").
- **–mail-type** Requests email notifications at the end of the job.
- **–mail-user** Uses the `$USER` environment variable to specify the email address where notifications will be sent. This assumes that the user's email is in the format `username@jhu.edu`.
- **–reservation** Associates the job with a reservation named `Training`.

2. The script loads the Intel compiler module with version 2022.2 using the `ml` command. This is done to ensure that the correct compiler environment is set up for compilation.
3. It echoes the compilation command that will be used (`mpiicc -qopenmp -o hello-mix.x hello-world-mix.c`). This is commented out because it's not actually compiling the code in the script, but you can uncomment it and run it outside the script.
4. Finally, it runs the mixed MPI/OpenMP program using the `mpirun` command with 2 MPI processes. The program is expected to use OpenMP for shared-memory parallelism.

To submit this mixed MPI/OpenMP job to the Slurm scheduler, save it to a file (e.g., `mpi_omp_job_script.sh`) and then submit it using the sbatch command:

```
[userid@local ~]$ sbatch mpi_omp_job_script.s
```

Note

The scheduler will allocate resources and run the mixed MPI/OpenMP program with the specified parameters.

ARCH Advanced Research Computing (https://www.arch.jhu.edu)

Address

5400 E Lombard St, Baltimore, MD 21224

Get Directions(https://goo.gl/maps/G4RtYwvoNqbz6QuC9)

Resources

FAQ(https://docs.arch.jhu.edu/en/latest/4_Support/FAQ.html)

ARCH Technical Documentation (https://docs.arch.jhu.edu/en/latest/)

Training

Training Sessions (https://www.arch.jhu.edu/support/training-education/)

Videos (https://www.arch.jhu.edu/support/training-education/)

Stay connected with alerts and newsletters

Email

SUBSCRIBE

ARCH is supported by:



(https://www.nsf.gov)



(https://www.defense.gov)

(https://www.jhu.edu)

Copyright 2023 | Advanced Research Computing at Hopkins (ARCH)